

UNIVERSIDAD TÉCNICA DE MACHALA

FACULTAD DE INGENIERÍA CIVIL

CARRERA DE TECNOLOGÍAS DE LA INFORMACIÓN

TEMA:

Desarrollo basado en especificaciones con GitHub Copilot

Gestor de incidencias en C#

CURSO

4TO. SEMESTRE PARALELO “A"

ESTUDIANTES RESPONSABLE

SANTIAGO NICOLAS BLACIO QUIMIS

CARDENAS FIGUEROA FRANK EMILSON

DOCENTE RESPONSABLE

ING. JOFFRE JEORWIN CARTUCHE CALVA

HORARIO DE CLASES

2-3 HORAS

PERIODO LECTIVO 2026-E2

MACHALA – EL ORO – ECUADOR

1 Introducción

El caso del primer parcial describe una empresa de software en Ecuador con pérdida de versiones, trabajo desorganizado, errores en producción, documentación insuficiente y falta de automatización. Es un escenario académico. No se dispone de mediciones de una empresa identificada, por lo que este trabajo no atribuye ahorros ni mejoras porcentuales a una organización real.

El objetivo general es aplicar un proceso de desarrollo en C# donde los requisitos guíen la generación de código, su comprobación y la colaboración. Los objetivos específicos son formular criterios observables, construir una aplicación mediante GitHub Copilot en Visual Studio y conservar evidencias del historial y de las verificaciones.

La indicación del docente establece las fases de requerimientos, especificación, plan técnico, implementación con IA, verificación y evolución (Ing. Cartuche, comunicación personal, 23 de octubre de 2026). Los estudiantes deben comprender y revisar el resultado generado. El documento Proyecto Parcial I - PA (s. f.) establece el problema y los productos de la actividad.

El producto es una aplicación local Windows Forms con .NET Framework 4.7.2. Registra y edita incidencias, permite buscarlas y controla sus estados. Conserva datos en XML. Las figuras del desarrollo reúnen capturas reales del código, los formularios y las comprobaciones de la aplicación. La revisión técnica asistida cuenta con evidencias; la decisión y aprobación personal del compañero siguen pendientes.

2 Desarrollo de los temas de investigación

2.1 Control de versiones y colaboración

Git permite conservar cambios en commits, comparar versiones y mantener ramas de trabajo. Pro Git describe el uso de ramas temáticas y líneas de integración para organizar cambios (Chacon & Straub, s. f.). En el caso, los commits separan requisitos, especificación, plan, entidades, persistencia, controlador, formularios, pruebas y automatización.

Merge integra historias. Rebase reorganiza commits sobre otra base y debe utilizarse con cuidado en ramas compartidas. Cherry-pick permite aplicar un cambio concreto. Para deshacer una modificación publicada se propone revertir mediante un nuevo commit, conservando el historial que utiliza el compañero.

Git Flow organiza funcionalidades, preparación de versiones y correcciones mediante distintas ramas. Trunk-based favorece integraciones frecuentes sobre la rama principal. Para dos integrantes se propone main estable y ramas breves con revisión. La demostración actual agrupa la funcionalidad en feature/gestor-incidencias. Es una simplificación académica y no demuestra todavía integración diaria.

Un pull request reúne cambios para revisión y discusión antes de integrarlos (GitHub, s. f.-d). Se publicaron main y feature/gestor-incidencias en el repositorio público indicado por el estudiante. El PR #1 permanece en borrador y no se fusionó. El paquete del 27 de octubre documenta verificaciones técnicas asistidas; la decisión y publicación de la revisión del compañero siguen pendientes.

Tabla 1. Comparación de flujos de trabajo

Aspecto	Git Flow	Trunk Based
Ramas	Desarrollo, funcionalidad, versión y corrección.	Integraciones frecuentes sobre una base estable.
Adecuación	Mantener varias versiones liberadas.	Equipo pequeño con revisión y CI.
Aplicación al caso	No se implementó el flujo completo.	Propuesta de ramas breves. La demostración actual agrupa una funcionalidad.

2.2 Desarrollo basado en especificaciones y GitHub Spec Kit

SDD utiliza la especificación como referencia para decidir qué construir y cómo aceptarlo. GitHub Spec Kit estructura el trabajo mediante especificación, planificación y tareas antes de la implementación (GitHub, s. f.-e). Este proyecto aplica ese enfoque con documentos versionados. Se consultó Spec Kit, pero no se afirma haber instalado ni ejecutado su CLI.

Los requerimientos describen necesidades y restricciones. La especificación fija el modelo, los contratos del controlador, las reglas de persistencia y el comportamiento de los formularios. El plan técnico divide el trabajo en tareas T01 a T08. Las pruebas comparan el resultado con esos contratos. Una evolución debe cambiar primero el criterio correspondiente y conservar el vínculo con sus modificaciones.

La Tabla 2 relaciona las fases indicadas por el docente con los archivos y cambios del caso. La Tabla 3 resume los contratos y criterios de aceptación. La Figura 1 documenta una solicitud real a Copilot. Una especificación actualizada permite evaluar el resultado, en vez de aceptar una respuesta de IA sin criterios.

Tabla 2. Fases de SDD y evidencias del caso

Fase	Evidencia	Referencia
Requerimientos	Alcance, RF, RNF y aceptación.	requirements.md, 7cc7511
Especificación	Modelo, contratos y formularios.	spec.md, 1554f64
Plan técnico	Capas y tareas T01-T08.	plan.md, ea58db6
Implementación	Generación y revisión por tarea.	Copilot en Visual Studio
Verificación	Pruebas y recorridos manuales.	Consola y Actions
Evolución	Prueba previa y corrección.	ead1743 y 074a9fc

2.3 Inteligencia artificial y prompt engineering

Copilot Agent puede trabajar sobre archivos y ejecutar tareas dentro de Visual Studio (Microsoft, s. f.-b). La implementación base y la prueba de regresión se generaron mediante solicitudes en el chat real de Copilot del proyecto. Las solicitudes se dividieron en cambios pequeños, se revisaron los archivos y se guardaron antes de compilar. El historial del chat permanece en el entorno del estudiante.

Un prompt adecuado identifica el documento de referencia, la tarea, las restricciones y el resultado que debe comprobarse (GitHub, s. f.-c). Ejemplo orientativo: implementar el controlador conforme a spec.md, conservar ID y fecha de creación al editar, persistir antes de confirmar éxito y rechazar transiciones inválidas. Este ejemplo resume el método y no pretende ser una transcripción literal del chat.

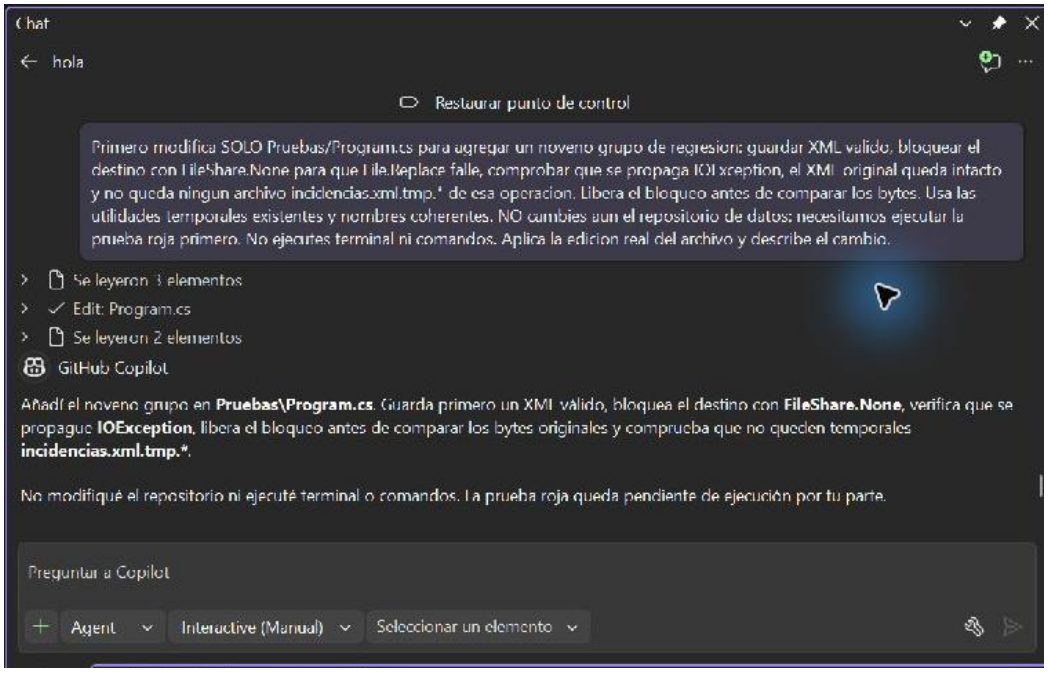


Figura 1. *Solicitud de una prueba de regresión a Copilot.* El prompt delimita el archivo, el bloqueo del XML y la conservación de los datos. Fuente: captura del proyecto, 25 de octubre de 2026.

La implementación base y la prueba de regresión se generaron con Copilot. Codex implementó la corrección posterior de temporales. Codex también coordinó solicitudes, verificó y preparó la documentación. La generación de la prueba y la implementación posterior del arreglo tienen procedencia distinta. La responsabilidad de comprender y aceptar el comportamiento permanece en los integrantes.

3 Aplicación práctica del modelo

La aplicación práctica sigue las seis fases de SDD indicadas por el docente. Los requisitos y contratos se versionaron antes de generar los componentes. La matriz del Anexo A relaciona los criterios de aceptación con sus verificaciones; los anexos B y C complementan los resultados de pruebas y colaboración.

3.1 Especificación del caso práctico

La entidad Incidencia contiene un identificador GUID, título, descripción, prioridad, estado y fechas UTC de creación y actualización. El título es obligatorio, se recorta y admite hasta 120 caracteres. La descripción es opcional y admite hasta 2000. Las prioridades válidas son Baja, Media y Alta.

Una incidencia nueva comienza Pendiente. Las transiciones autorizadas son Pendiente a EnProceso, EnProceso a Resuelta y Resuelta a EnProceso. Cualquier otra transición debe rechazarse sin cambiar memoria ni almacenamiento. El usuario puede editar título, descripción y prioridad, conservando identificador y fecha de creación.

Criterio de aceptación: dada una incidencia Pendiente, cuando se solicita pasar directamente a Resuelta, el controlador informa el error y conserva su estado previo. Otro criterio exige que un fallo de guardado no aparezca como una operación exitosa ni altere la colección confirmada.

Tabla 3. Contratos y criterios de aceptación

Contratos	**Aceptación**	**Resultado requerido**
Registro y límites	CA-01 a CA-05	Alta válida, título obligatorio y enums definidos.
Edición	CA-06	Conservar ID y creación; actualizar datos y fecha.
Consulta y filtros	CA-07 a CA-08	Combinar texto, prioridad y estado.
Transiciones	CA-09 a CA-10	Aceptar solo destinos autorizados.
Persistencia e integridad	CA-11 a CA-14	Recarga, XML protegido y error de guardado explícito.

El archivo XML ausente representa una colección vacía. Un archivo presente pero corrupto, con versión no admitida, IDs duplicados o fechas inválidas produce un error explícito y

bloquea escrituras durante la sesión. El sistema no interpreta un archivo ilegible como vacío. No incluye autenticación, eliminación, acceso multiusuario ni una base de datos de producción.

3.2 Arquitectura y formularios

La solución separa Entidades, Controladores, Datos y Formularios. Incidencia y sus enumeraciones representan el dominio. ControladorIncidencias valida entradas, controla transiciones y coordina la persistencia. IRepositorioIncidencias define el contrato de almacenamiento y RepositorioXmlIncidencias lo implementa. Esta distribución coincide con los conceptos de las prácticas de clase proporcionadas como guía.

FrmPrincipal presenta el DataGridView de solo lectura, búsqueda y filtros. FrmIncidencia permite registrar y editar, y utiliza ErrorProvider para señalar entradas inválidas. FrmDetalle muestra identidad, contenido, fechas y las siguientes transiciones permitidas. Los tres tienen archivos Designer.cs editables. Las reglas no dependen de eventos particulares de los controles.

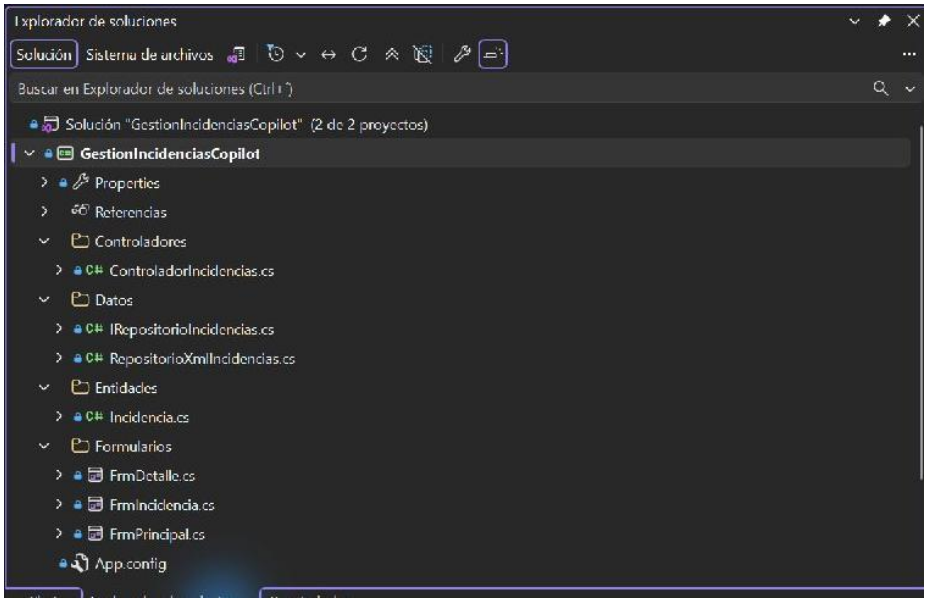


Figura 2. *Organización de la solución en Visual Studio.* Las carpetas separan entidades, controladores, datos y formularios con sus archivos de diseño. Fuente: captura del proyecto, 25 de octubre de 2026.

El controlador devuelve copias y confirma un cambio después de guardarlo. El repositorio valida XML versionado y fechas UTC. La escritura usa un temporal único, Flush(true), Move o File.Replace. El finally de la corrección limpia solo el temporal creado por esa operación y evita ocultar el error original. El almacenamiento es local, sin coordinación multiusuario.

3.3 Demostración y verificación manual

La Figura 3 muestra el caso de demostración persistido. Las figuras 4 y 5 muestran la validación del formulario de alta y el detalle de la incidencia. Las capturas corresponden a la ejecución de la aplicación en Windows.

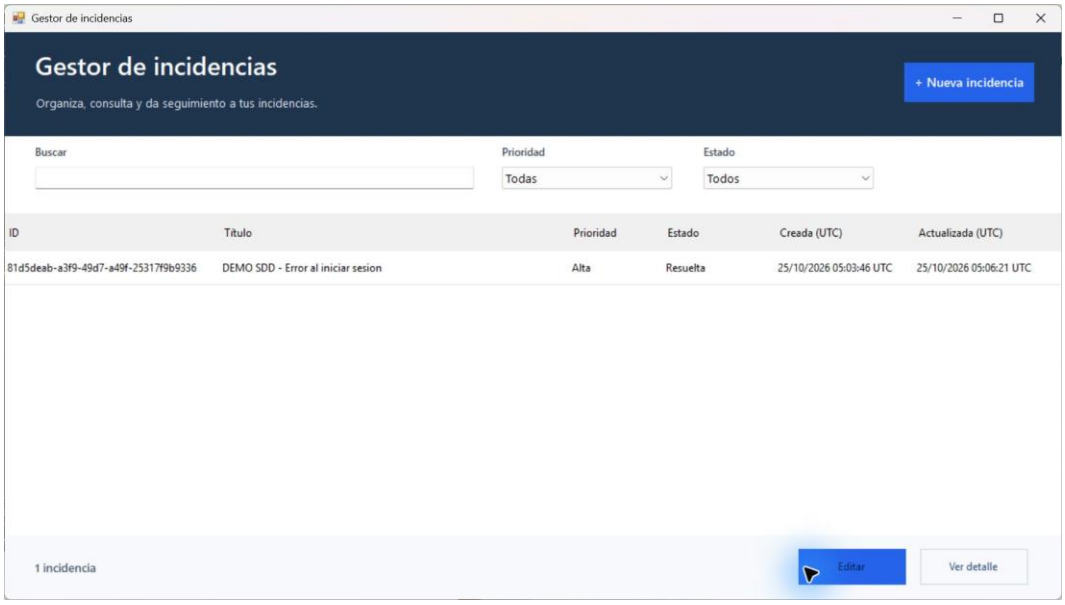


Figura 3. *Listado principal de incidencias.* La incidencia de demostración conserva prioridad Alta, estado Resuelta e identificación después del reinicio. Fuente: captura del proyecto, 25 de octubre de 2026.

Se rechazó un título vacío y se registró DEMO SDD - Error al iniciar sesion con prioridad Alta. La aplicación permitió pasar de Pendiente a EnProceso y luego a Resuelta. La edición conservó el identificador 81d5deab-a3f9-49d7-a49f-25317f9b9336 y la fecha de

creación. Se comprobó la búsqueda por descripción sin distinguir mayúsculas y la combinación de filtros con y sin coincidencias.

Tras cerrar y reabrir la aplicación en Release, se conservaron identificador, prioridad, estado y fechas. Las capturas adjuntas documentan listado, detalle y persistencia después del reinicio. La verificación manual documentada corresponde a la compilación Release.

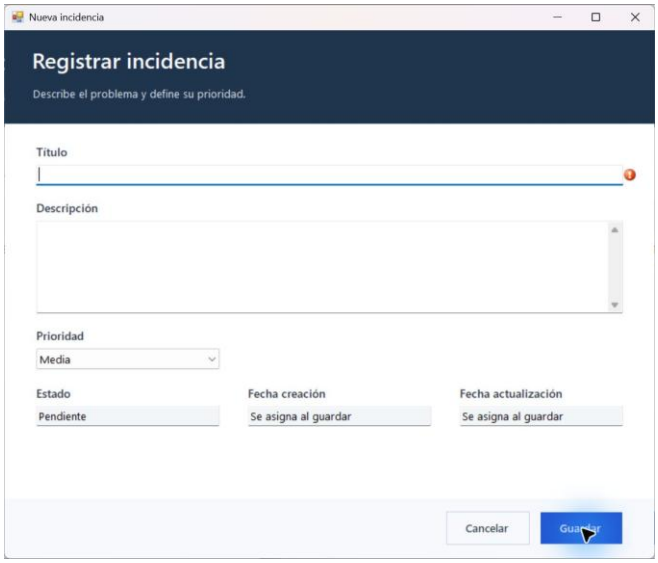


Figura 4. *Validación del título obligatorio.* El indicador de error aparece al guardar sin título y el formulario permite corregirlo. Fuente: captura del proyecto, 25 de octubre de 2026.

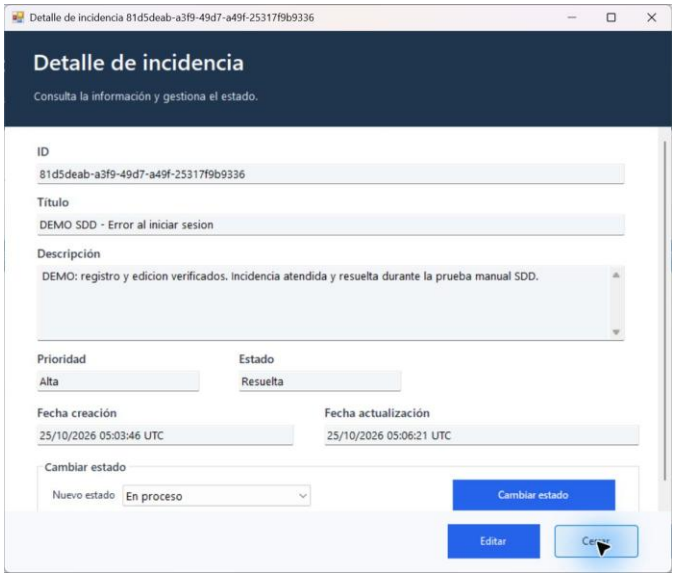


Figura 5. *Consulta de una incidencia resuelta.* El detalle muestra los datos persistidos y ofrece En proceso como transición permitida. Fuente: captura del proyecto, 25 de octubre de 2026.

3.4 DevOps y automatización con GitHub Actions

GitHub Actions permite repetir compilación y verificaciones a partir de eventos del repositorio (GitHub, s. f.-b). El workflow del caso usa Windows y MSBuild, compila el proyecto de pruebas con su referencia a la aplicación, ejecuta las aserciones y conserva binarios Release. Se activa en main, ramas feature y pull requests hacia main.

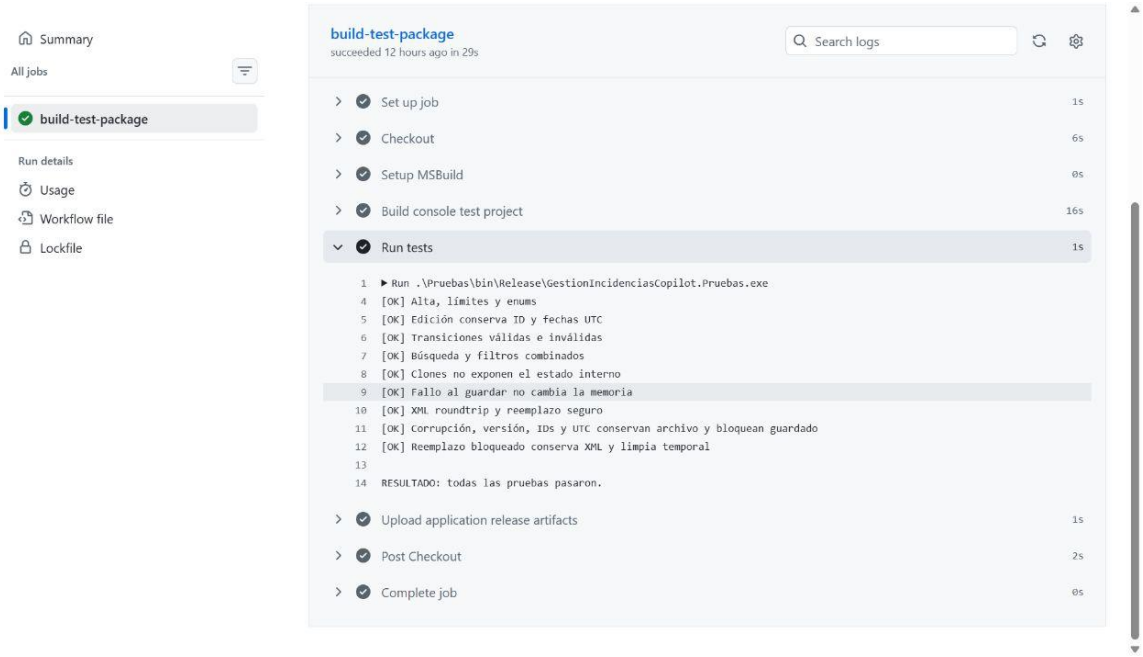


Figura 6. *Pruebas ejecutadas en GitHub Actions.* El paso Run tests del run 40204708554 registra nueve grupos aprobados. Fuente: captura del proyecto, 25 de octubre de 2026.

El verificador devuelve 0 cuando aprueba y 1 ante fallos, por lo que puede detener el pipeline. El registro detallado de la evolución de temporales se incluye en el Anexo B. El artefacto permite distribuir la compilación para evaluación. No existe despliegue productivo. Para WinForms se propone distribución versionada y firmada, entorno piloto y reversión a una versión anterior.

3.5 Calidad y evolución con prueba previa

TDD plantea escribir una prueba que falle, implementar lo necesario y refactorizar conservando el resultado. Las buenas prácticas de pruebas de .NET favorecen casos comprensibles que protejan comportamientos útiles frente a regresiones (Microsoft, s. f.-a). En este proyecto las pruebas se generaron después de la aplicación. No se conserva un ciclo rojo-verde-refactor previo y no corresponde afirmar que se aplicó TDD completo.

El proyecto Pruebas es un ejecutable de aserciones sin paquetes externos. No utiliza xUnit, MSTest ni un adaptador de dotnet test. Devuelve código cero cuando todos los grupos pasan y uno ante cualquier fallo. Cada grupo contiene varias aserciones. Nueve grupos aprobados no equivalen a nueve pruebas individuales ni a cobertura total.

Tabla 4. Grupos de verificaciones automatizadas

Grupo	Comportamiento comprobado
1	Alta, límites y enumeraciones.
2	Edición conserva identidad y fechas UTC.
3	Transiciones válidas e inválidas.
4	Búsqueda y filtros combinados.
5	Copias no exponen estado interno.
6	Fallo al guardar conserva memoria.
7	XML, recarga y reemplazo seguro.
8	Corrupción, versión, IDs y UTC protegen el archivo.
9	Destino bloqueado conserva XML y limpia temporal propio.

Cada grupo contiene varias aserciones y no equivale a una prueba individual. No se midió cobertura. La evolución de temporales conserva primero una prueba que falla (ead1743) y después el arreglo aprobado (074a9fc). No acredita TDD completo de la aplicación base ni una

refactorización adicional. El Anexo B conserva las salidas de ambas ejecuciones y sus commits de referencia.

3.6 Gestión del proyecto y colaboración

GitHub Projects organiza Issues y PR en vistas de tabla y tablero (GitHub, s. f.-a). Se creó el Project Gestion de incidencias - SDD y Copilot y el milestone Primer parcial - SDD y Copilot. La Figura 7 muestra el estado real de sus elementos. Los Issues contienen criterios de aceptación y evidencia.

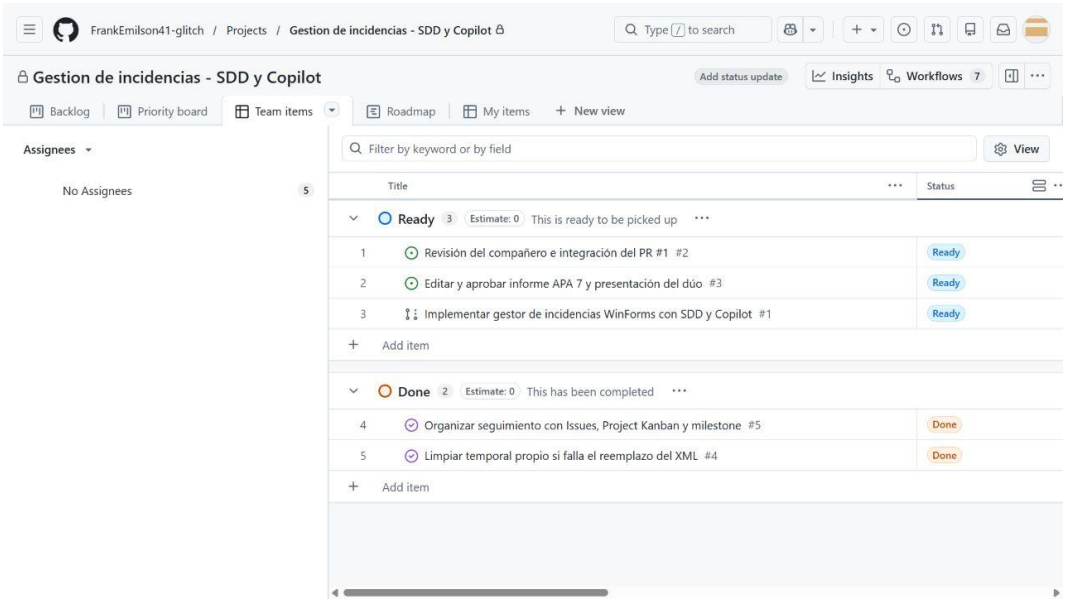


Figura 7. *Seguimiento del trabajo en GitHub Projects.* La captura distingue tres elementos Ready y dos Done. El PR y la revisión permanecen pendientes. Fuente: captura del proyecto, 25 de octubre de 2026.

El 27 de octubre se recibió un paquete de revisión técnica asistida para Santiago Nicolás Blacio Quimis. Documenta compilación Release, nueve grupos aprobados y un recorrido de formularios sobre el commit 6042e94. No registra cambios de código ni una revisión publicada. La decisión personal del compañero sigue pendiente. El Anexo C presenta los resultados y las capturas seleccionadas (Revisión técnica asistida del PR 1, 2026).

La existencia de un tablero creado al cierre no demuestra su uso durante toda la construcción. Para un siguiente ciclo se propone abrir cada tarea antes de generar código y actualizar su estado con evidencia.

4 Análisis crítico

SDD hace explícito el criterio con el que se acepta una modificación. Una petición general de gestionar incidencias no define el manejo de corrupción o transiciones inválidas. Los contratos permitieron revisar esos escenarios. Sin embargo, una especificación equivocada puede producir código y pruebas coherentes entre sí que tampoco resuelvan la necesidad original. La revisión debe comenzar por la necesidad y los ejemplos negativos.

La evolución de temporales distingue entre el comportamiento habitual y los fallos. El guardado normal funcionaba y protegía el XML original, pero dejaba un temporal al bloquear el reemplazo. La prueba negativa reprodujo el defecto antes de cambiar Datos. El arreglo intenta eliminar exclusivamente el temporal propio y conserva el error original si la limpieza también falla. No promete borrar un archivo que el sistema operativo impida eliminar.

Nueve grupos aprobados no representan cobertura completa ni pruebas de carga. El pipeline hace repetible lo que se ha definido, pero no descubre por sí solo requisitos omitidos. La advertencia de Actions requiere evaluar sus versiones en una evolución controlada. Un resultado verde tampoco sustituye la revisión de otra persona.

XML simplifica la demostración local, pero limita la concurrencia y la atribución de cambios. Antes de un uso empresarial se requieren autenticación, almacenamiento transaccional, respaldos y distribución controlada. Esas ampliaciones deben incorporarse primero a las especificaciones. El caso no demuestra ahorros o mejoras porcentuales de una empresa real.

5 Propuesta de solución para la empresa

Se propone comenzar por un repositorio compartido, requisitos comprobables y revisión de cambios. Cada necesidad debe corresponder a un Issue y un criterio. Después se solicitarán cambios acotados a la IA, se guardará el historial y se ejecutará CI. Los binarios de una ejecución aprobada serán la base de un piloto, con posibilidad de regresar a una versión anterior.

Tabla 5. Adopción gradual propuesta para la empresa

Etapa propuesta	**Acción y condición de avance**
Definir	Acordar necesidad, alcance y aceptación con usuarios.
Construir	Dividir tareas y generar cambios pequeños con IA.
Verificar	Compilar, probar fallos y revisar con el compañero.
Pilotar	Distribuir versión identificable y conservar reversión.
Evaluar	Registrar defectos y cambios vinculados antes de ampliar.

En dos semanas se propone dedicar la primera a investigación, especificación y construcción por tareas, y la segunda a pruebas, revisión y exposición. Es una planificación sugerida, no un registro de horas ejecutadas. Santiago puede exponer problema, Git y SDD; Frank puede explicar arquitectura y demostración. Ambos deben comprender el comportamiento completo.

Los indicadores del piloto serían cambios vinculados a requisitos, revisiones realizadas, ejecuciones de CI y defectos encontrados antes de integrar. Estos indicadores requieren una medición durante el piloto. Para producción deben definirse responsabilidades de soporte, respaldo y seguridad, además del mecanismo de distribución.

6 Reflexiones del equipo

¿Qué problema resuelve Git en el desarrollo real?

Conserva un historial recuperable y permite trabajar en ramas. En este caso ayuda a identificar el cambio que incorporó una regla o introdujo un defecto, sin depender de carpetas duplicadas. El remoto aporta un punto compartido, acompañado de políticas de acceso y respaldo.

¿Por qué DevOps es importante en la actualidad?

Permite repetir compilación y comprobaciones de forma consistente antes de integrar o distribuir. El workflow conserva evidencia que otra persona puede consultar. Su valor depende de que las pruebas representen necesidades útiles y de que exista una decisión de revisión y entrega.

¿Cómo la IA está cambiando la programación?

Genera propuestas a partir de contexto e instrucciones. El trabajo humano incluye definir criterios, comprender el código y comprobar errores. Los integrantes siguen siendo responsables de justificar que el resultado satisface la necesidad. Una respuesta convincente debe contrastarse con evidencia.

¿Qué herramienta fue más útil y por qué?

Para construir el caso, GitHub Copilot en Visual Studio fue especialmente útil porque generó entidades, controlador, persistencia y formularios a partir de las especificaciones dentro del mismo entorno. La Figura 1 muestra una solicitud acotada de prueba. Git y Actions permitieron conservar y comprobar los cambios. La utilidad de Copilot depende de aportar contexto y verificar cada resultado frente a la especificación.

7 Conclusiones

El proyecto implementa una aplicación Windows Forms en C# con formularios editables, reglas separadas y almacenamiento XML. Los documentos y commits vinculan necesidades, contratos, planificación e implementación. Las figuras del desarrollo permiten inspeccionar resultados concretos y relacionarlos con los contratos del sistema.

Copilot generó la implementación base y la prueba de regresión. La corrección posterior de temporales se implementó con Codex y conserva evidencia de prueba fallida y aprobada. Los nueve grupos pasan localmente y en GitHub Actions. La procedencia de cada intervención se mantiene explícita.

La propuesta empresarial integra especificaciones, Git, revisión y automatización mediante una adopción gradual. Requiere mantener los criterios, diseñar pruebas útiles y contar con personas capaces de revisar el resultado. El prototipo demuestra un flujo local, sin atribuir autenticación, despliegue productivo o concurrencia como funciones existentes.

La práctica cuenta con aplicación C#, ramas, PR, pipeline y documentación. El paquete del 27 de octubre acredita verificaciones técnicas asistidas; la decisión y publicación de la revisión del compañero siguen pendientes. Spec Kit fue investigado sin ejecutar su CLI. No se atribuye TDD completo a la aplicación base.

Referencias

Chacon, S., & Straub, B. (s. f.). *Branching workflows*. https://git-scm.com/book/en/v2/Git-Branching-Branching-Workflows

GitHub. (s. f.-a). *About Projects*. https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects

GitHub. (s. f.-b). *Building and testing .NET*. https://docs.github.com/en/actions/tutorials/build-and-test-code/net

GitHub. (s. f.-c). *Prompt engineering for GitHub Copilot*. https://docs.github.com/en/copilot/concepts/prompting-and-context/prompt-engineering

GitHub. (s. f.-d). *Pull requests*. https://docs.github.com/en/pull-requests/reference/pull-requests

GitHub. (s. f.-e). *Spec Kit [Repositorio de software]*. https://github.com/github/spec-kit

Microsoft. (s. f.-a). *Unit testing best practices*. https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-best-practices

Microsoft. (s. f.-b). *Use agent mode in Visual Studio*. https://learn.microsoft.com/en-us/visualstudio/ide/copilot-agent-mode

Proyecto Parcial I - PA. (s. f.). [Guía académica de Programación Avanzada en C# proporcionada por el docente].

Revisión técnica asistida del PR 1. (2026, 27 de octubre). [Paquete de evidencias PARA_FRANK_PR1_COMPLETO.zip proporcionado por Santiago Nicolás Blacio Quimis].

Anexo A

Trazabilidad de requisitos y verificación

Esta matriz complementa los contratos resumidos en la Tabla 3. Vincula los requisitos RF, los criterios CA y las comprobaciones del proyecto; permite localizar qué evidencia corresponde a cada comportamiento.

Tabla A1. Relación entre requisitos y verificaciones

Requisitos	Criterios	Verificación documentada
RF-01 a RF-04	CA-01 a CA-05	Grupo 1: alta, límites y enumeraciones. Figura 4: título vacío rechazado.
RF-05	CA-06	Grupo 2: conservar ID y fecha de creación al editar.
RF-06	CA-07	Recorrido manual de listado y detalle. Figuras 3 y 5.
RF-07 y RF-08	CA-08	Grupo 4: búsqueda y filtros combinados.
RF-09	CA-09 y CA-10	Grupo 3: destinos válidos e inválidos.
RF-10	CA-11 y CA-14	Grupos 6, 7 y 9: fallo de guardado, recarga XML y destino bloqueado. Reinicio manual.
RF-11 y RF-12	CA-12 y CA-13	Grupo 8: corrupción, versión, identificadores y fechas.

La especificación completa está en specs/001-incidencias. Los documentos requirements.md, spec.md y plan.md se conservan en los commits 7cc7511, 1554f64 y ea58db6, anteriores a los componentes de la aplicación.

Los grupos 5 y 6 verifican además que las copias no expongan el estado interno y que un fallo de persistencia no confirme cambios en memoria. La matriz identifica evidencia existente; no constituye una medición de cobertura.

Anexo B

Registros de la prueba de regresión de temporales

La prueba bloquea un XML válido y solicita un guardado. Comprueba que se informe el fallo, se conserven los bytes originales y no quede un temporal creado por la operación. Se reproducen las salidas de los registros del repositorio.

Antes de la corrección commit ead1743

```
[OK] Alta, límites y enums
[OK] Edición conserva ID y fechas UTC
[OK] Transiciones válidas e inválidas
[OK] Búsqueda y filtros combinados
[OK] Clones no exponen el estado interno
[OK] Fallo al guardar no cambia la memoria
[OK] XML roundtrip y reemplazo seguro
[OK] Corrupción, versión, IDs y UTC conservan archivo y bloquean guardado
[FALLO] Reemplazo bloqueado conserva XML y limpia temporal —
InvalidOperationException: Quedó un temporal incidencias.xml.tmp.* después del fallo.
RESULTADO: 1 prueba(s) fallaron.
```

Después de la corrección commit 074a9fc

```
[OK] Alta, límites y enums
[OK] Edición conserva ID y fechas UTC
[OK] Transiciones válidas e inválidas
[OK] Búsqueda y filtros combinados
[OK] Clones no exponen el estado interno
[OK] Fallo al guardar no cambia la memoria
[OK] XML roundtrip y reemplazo seguro
[OK] Corrupción, versión, IDs y UTC conservan archivo y bloquean guardado
[OK] Reemplazo bloqueado conserva XML y limpia temporal
RESULTADO: todas las pruebas pasaron.
```

Fuente: docs/03-verificacion/evidencias/prueba_temporales_roja.txt y prueba_temporales_verde.txt. La implementación base recibió pruebas después del código; estos registros acreditan una prueba previa para esta evolución concreta.

Anexo C

Revisión técnica asistida del compañero

El paquete recibido el 27 de octubre de 2026 documenta la revisión del commit 6042e943b0bd5338c0a481c66f8d3b3564aeff87. Contiene doce capturas de formularios, registros de compilación y pruebas, XML de demostración y un borrador de observaciones. Las 42 entradas del manifiesto coinciden en tamaño y SHA-256 con los archivos recibidos. **Compilación.** El log de MSBuild registra Release correcto, cero errores y cero advertencias.

Pruebas. El primer intento presenta dos errores de permisos en temporales. El segundo usa TEMP/TMP en una carpeta escribible y registra nueve grupos OK, con ExitCode=0. El paquete no atribuye los errores iniciales a un defecto funcional confirmado.

Formularios. Se documentan validación del título, alta, transiciones, edición conservando identidad y creación, filtros combinados y persistencia tras reiniciar.

Cambios y decisión. El resumen declara que no se modificó código ni se crearon commits. No incluye una revisión publicada ni la aprobación de Santiago. No se obtuvo una captura de la compilación o de la consola de pruebas; se conservan sus registros originales.

Observaciones propuestas

• Precisar el texto de edición de FrmIncidencia.cs: se conserva la fecha de creación, mientras que la actualización cambia al guardar.

• Ampliar las pruebas XML para GUID vacío o mal formado, fechas imposibles, actualización anterior a creación y enumeraciones inválidas. Son casos pendientes de prueba explícita, no defectos confirmados de la validación.

• Actualizar la descripción del PR: todavía indica ocho grupos y Actions pendiente, según el estado guardado en el paquete. La revisión enlaza el run 40208728758 del commit revisado como aprobado.

Alcance de la evidencia

Los resultados anteriores proceden del paquete recibido y no constituyen una nueva ejecución en el equipo de Frank. La consulta pública del PR no se pudo recuperar durante esta revisión. El índice contiene una inconsistencia horaria: 17:09:48Z equivale a 12:09:48 en UTC−05:00. Se conservan los JSON originales y se utilizan sus fechas UTC, sin alterar capturas ni registros.

Capturas seleccionadas de la revisión

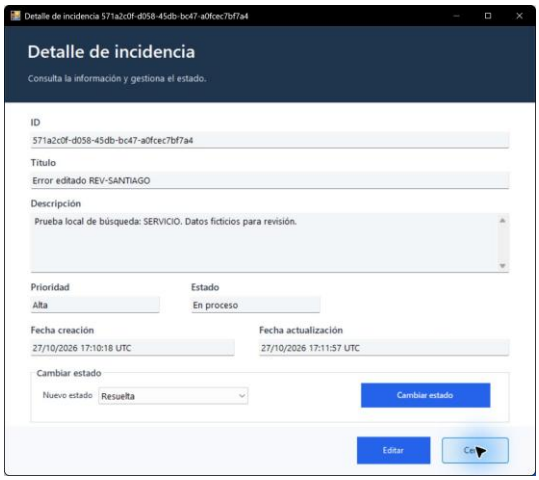


Figura 8. *Edición comprobada durante la revisión.* La incidencia REV-SANTIAGO conserva ID y creación, con prioridad Alta y una actualización posterior. Fuente: paquete de revisión, 27 de octubre de 2026.

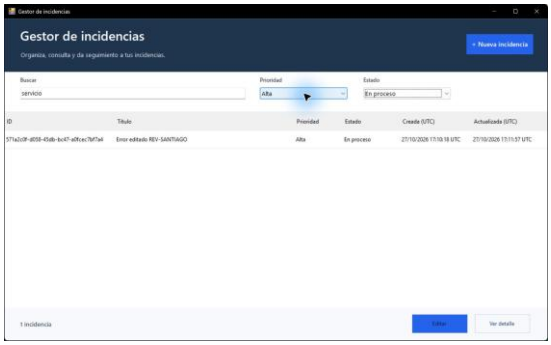


Figura 9. *Búsqueda y filtros conjuntos.* El texto servicio, prioridad Alta y estado En proceso producen una coincidencia. Fuente: paquete de revisión, 27 de octubre de 2026.

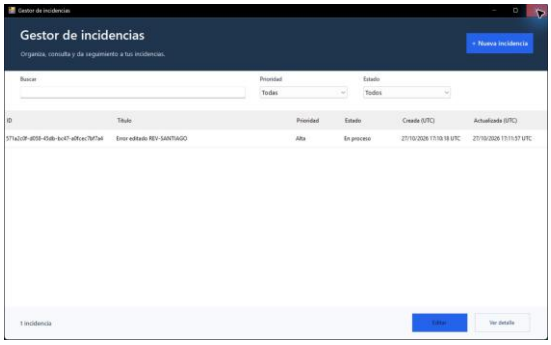


Figura 10. *Persistencia tras reiniciar.* Se recupera el registro editado. Los archivos de hash antes y después contienen el mismo SHA-256. Fuente: paquete de revisión, 27 de octubre de 2026.